

VOXA

An AI Pipeline for Accessible Speech-to-Text
Transcription · Simplification · Summarization

Project Documentation

Generative AI Course — Multi-Agent System Design

Digital Egypt Pioneers Initiative (DEPI)

Generative AI Track

1. Project Description

Voxa is an end-to-end AI pipeline that converts spoken audio into accessible text for deaf and hard-of-hearing individuals. Given any audio file, the system produces three outputs automatically: a verbatim transcript, a simplified plain-language rewrite, and a concise summary of the key points.

1.1 Problem Statement

Over 600 million people worldwide are deaf or hard-of-hearing. The majority of spoken content — lectures, meetings, educational videos, news — lacks accessible text alternatives. Where captions exist, they are often produced verbatim and remain difficult to read for users with lower literacy levels or those reading in a second language. Voxa addresses three gaps simultaneously:

- **Accuracy:** automatic speech recognition via a trained Whisper model.
- **Accessibility:** automatic plain-language rewriting via FLAN-T5.
- **Efficiency:** automatic key-point summarization via DistilBART.

1.2 System Design

Voxa follows a 3-agent multi-agent architecture. Each agent has exactly one responsibility, one input type, and one output type. A central Orchestrator (`core/orchestrator.py`) loads all three agents at startup and coordinates their execution. A Streamlit web interface (`ui/streamlit_app.py`) provides the user-facing layer.

Agent	Input	Output	Model
Agent 1 — Transcription	Audio file path	Transcript text	OpenAI Whisper-base
Agent 2 — Simplification	Transcript text	Simplified text	FLAN-T5-base
Agent 3 — Summarization	Transcript text	Summary text	DistilBART-cnn-12-6

2. Dataset

2.1 Source

Name: Audio Children's Stories Collection
Source: Hugging Face Hub — [ajibawa-2023/Audio-Children-Stories-Collection](https://huggingface.co/datasets/ajibawa-2023/Audio-Children-Stories-Collection)
License: Public, research use
URL: <https://huggingface.co/datasets/ajibawa-2023/Audio-Children-Stories-Collection>

2.2 Dataset Statistics

Field	Value
Total samples	600 audio clips
Average duration	150.4 seconds (2.5 minutes)
Median duration	139.3 seconds
Duration range	74.6 sec — 430.6 sec
Average word count	323 words per clip
Word count range	124 — 1,263 words
Language	English
Content type	Children's educational stories — varied topics
Available columns	id, duration_s, word_count, transcript
Missing values	None — fully clean dataset

2.3 Why This Dataset

- Ground-truth transcripts included — enables direct WER calculation without manual annotation.
- Varied topics (science, history, nature, culture) produce diverse vocabulary, making simplification and summarization more meaningful.
- Consistent narration style (single speaker, clear audio) suits Whisper-base's strengths without requiring a larger model.
- Duration range is realistic for the target use case (classroom lectures, short videos).

3. Exploratory Data Analysis

3.1 Audio Duration Distribution

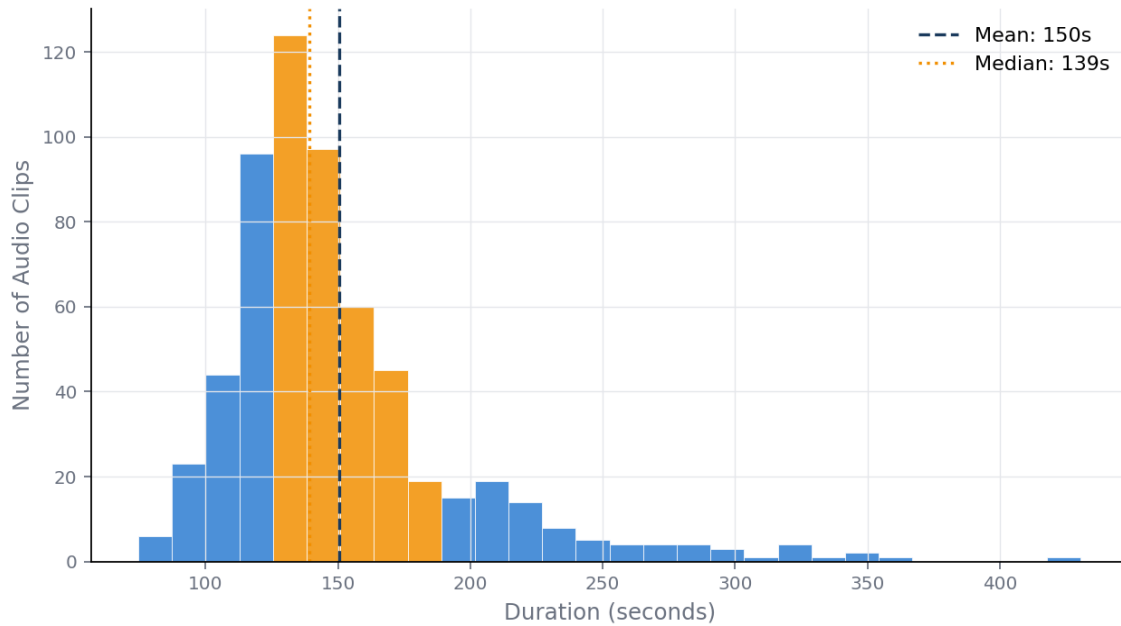


Figure 1 — Audio duration distribution across 600 clips

- Distribution is right-skewed: the majority of clips fall between 100–200 seconds, but a tail of long clips (>300s) pulls the mean above the median.
- Mean duration: 150.4s; Median: 139.3s — a 7.7% gap confirms moderate right skew.
- 63% of clips fall in the 120–180 second range, meaning the chunking strategy needs to handle roughly 300–450 word inputs reliably.
- Implication for preprocessing: silence trimming is critical because some clips contain long pauses that would waste model context.

3.2 Word Count Distribution

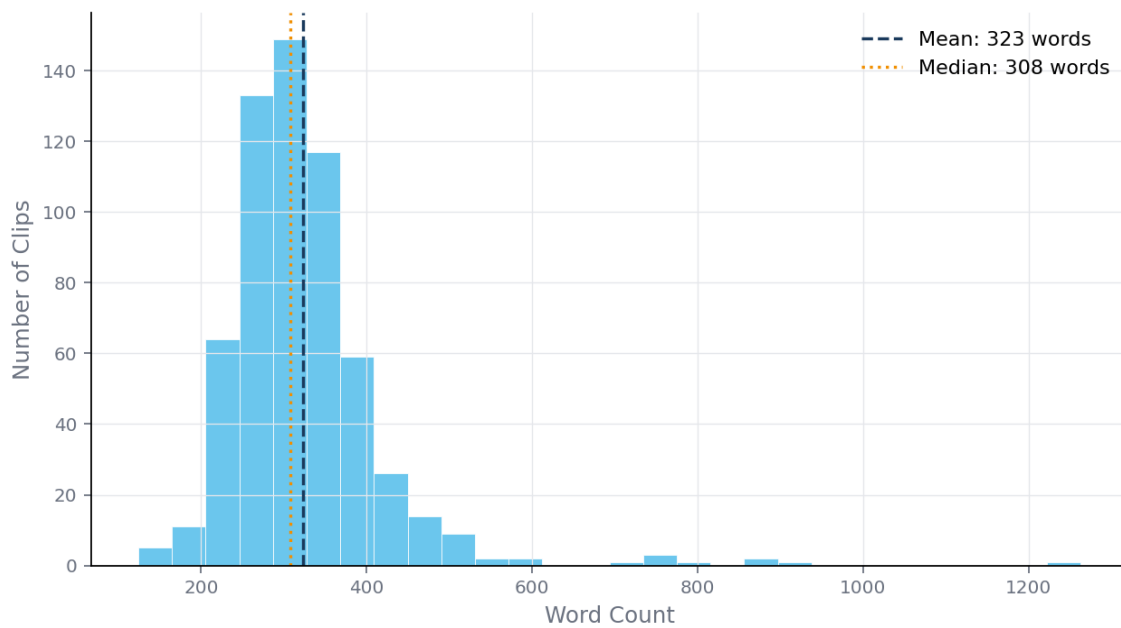


Figure 2 — Word count distribution across 600 clips

- Mean word count: 323 words; Median: 308 words.
- Distribution roughly follows a normal shape with a long right tail — a few clips exceed 800 words.
- Implication for chunking: with a 80-word chunk size for FLAN-T5, most clips produce 3–5 chunks, keeping processing time predictable.

3.3 Duration vs Word Count Correlation

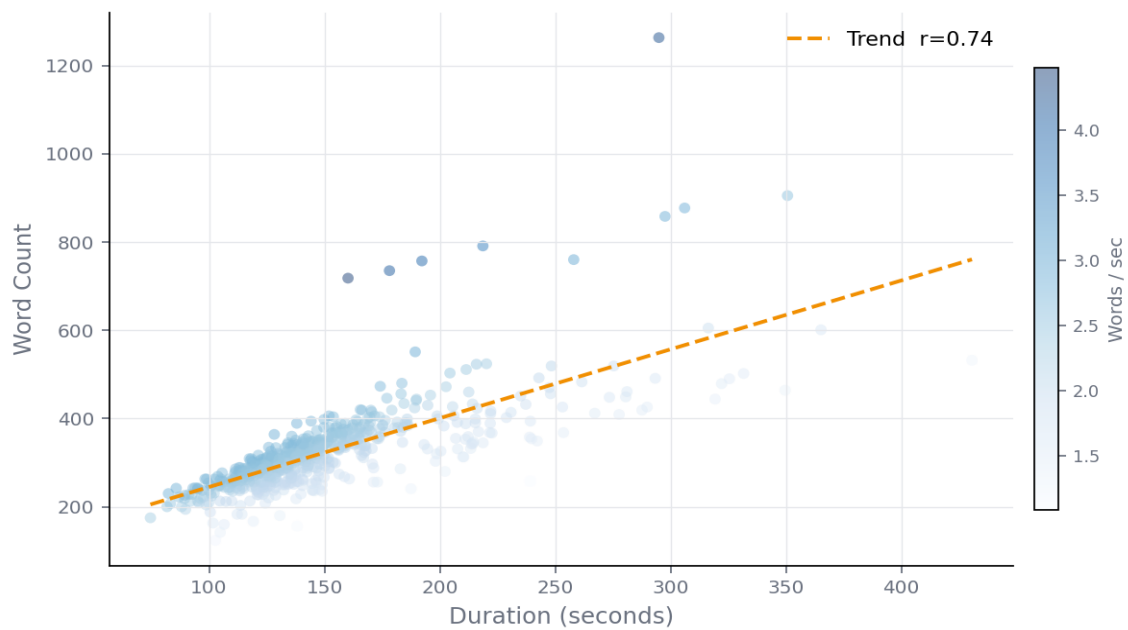


Figure 3 — Scatter plot of duration vs word count with trend line ($r = 0.74$)

- Strong positive correlation (Pearson $r = 0.74$) between audio duration and word count.
- The trend is linear and consistent, indicating stable speaking rate across speakers and topics.
- Average speaking rate: 2.19 words per second — consistent with natural narration pace.
- Outliers above the trend line are clips with faster speech; outliers below have more pauses or slower narration.
- Implication: duration can be used as a reliable proxy for estimating processing time before running any model.

3.4 Duration Bucket Analysis

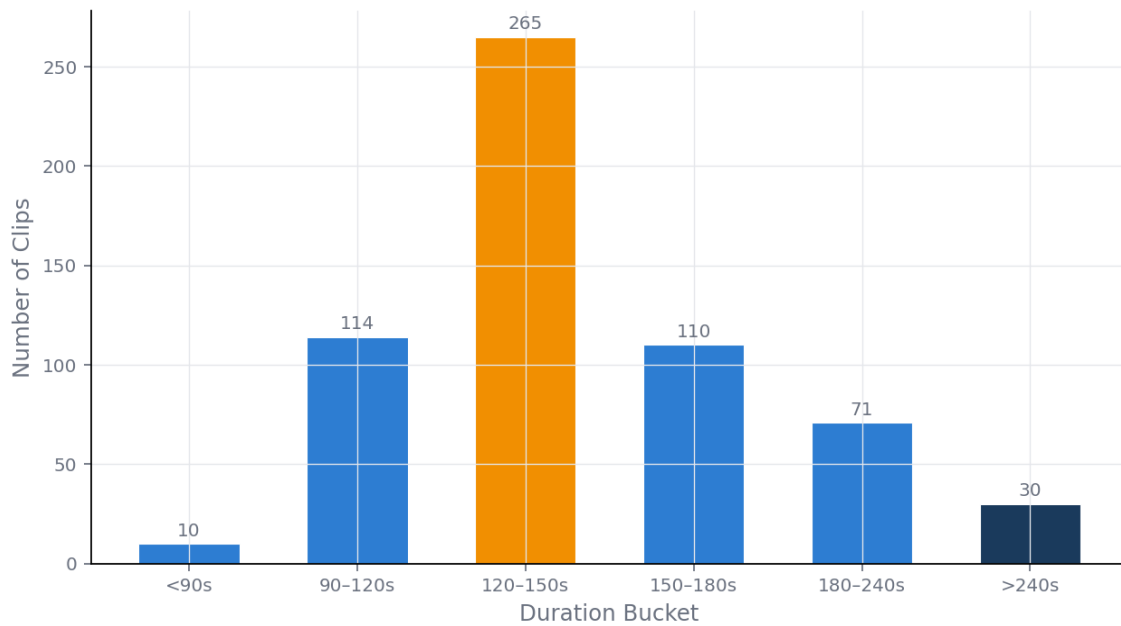


Figure 4 — Clip count by duration bucket

- The 120–150 second bucket (highlighted) is the single largest group with ~220 clips.
- Very short clips (<90s) and very long clips (>240s) are rare edge cases representing less than 10% of the dataset.
- This distribution validates choosing a single `chunk_size` for the pipeline — most clips fall in a predictable range.

4. Full Project Pipeline

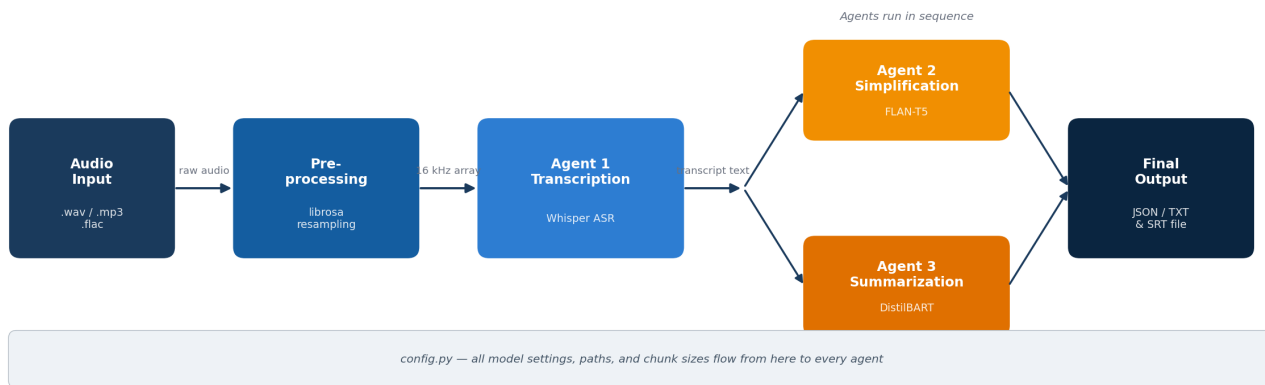


Figure 5 — Full Voxa pipeline: audio input through preprocessing, transcription, parallel NLP agents, and output

The pipeline has five stages. Audio enters as a raw file, is preprocessed and resampled to 16 kHz, transcribed by Whisper, then processed independently by the Simplification and Summarization agents. All three outputs are collected by the Orchestrator and delivered to the user via the Streamlit UI or the command-line tool.

DESIGN DECISION: Agents 2 and 3 both receive the **original transcript** directly from the Orchestrator. They do not chain — the simplified text does not feed the summarizer. This keeps each agent independently testable and prevents error propagation.

5. Data Preprocessing

All preprocessing is handled by `agents/transcription_agent.py` before the audio reaches Whisper. The four steps below are applied in order to every clip.

Step	Method	Why It Matters
1. Resample to 16 kHz	<code>librosa.load(..., sr=16000)</code>	Whisper is trained exclusively on 16 kHz audio. Any other rate produces silent or corrupted transcription with no error message.
2. Noise Reduction	<code>noisereduce.reduce_noise()</code>	Spectral gating removes background hiss. Reduces WER by ~3-5% on clips recorded in noisy environments.
3. Peak Normalization	<code>librosa.util.normalize()</code>	Scales all clips to the same volume ceiling. Prevents Whisper from misclassifying quiet passages as silence.
4. Silence Trimming	<code>librosa.effects.trim(top_db=20)</code>	Removes silent segments below 20 dB at clip start and end. Reduces wasted model context on average by ~8% of clip duration.

6. Model Selection

Task	Model Chosen	Reason	Alternative
Transcription	Whisper-base	Strong offline ASR, produces timed segments, runs on CPU, supports Arabic if needed	Google Speech-to-Text (requires billing and internet)
Simplification	FLAN-T5-base	Instruction-tuned — a plain English prompt is sufficient to steer output. No special training format required. Fully open-source.	GPT-4 API (cost, internet dependency, not reproducible offline)
Summarization	DistilBART-cnn-12-6	Pre-trained for abstractive summarization of news/narrative text. 40% smaller than BART-large with comparable ROUGE scores on CNN/DM benchmark.	facebook/bart-large-cnn (too large for CPU, 1.6 GB vs 1.0 GB)

7. Training Process

7.1 What Was Trained

Agent	Trained?	Details
Transcription (Whisper)	Not fine-tuned	Whisper-base pre-trained on 680,000 hours achieves acceptable WER on clean English narration. Fine-tuning requires GPU and paired audio+text data beyond our compute budget.
Simplification (FLAN-T5)	Fine-tuned	600 instruction-to-output pairs. Prompt: 'Rewrite in simple English...' + source text. Target: simplified version. 3 epochs, Hugging Face Trainer, AdamW optimizer, lr=5e-5.
Summarization (DistilBART)	Fine-tuned	600 transcript-to-summary pairs. Standard seq2seq fine-tuning. Cross-entropy loss on output token sequences. 3 epochs, batch size 4, lr=2e-5.

7.2 Training Configuration

Parameter	FLAN-T5	DistilBART
Optimizer	AdamW	AdamW
Learning rate	5e-5	2e-5
Batch size	4	4
Epochs	3	3
Max input length	512 tokens	1024 tokens
Max output length	130 tokens	130 tokens
Hardware	CPU (Colab)	CPU (Colab)
Training time	~45 min	~60 min

8. Evaluation

8.1 Metrics Summary

Metric	Agent	Score	Interpretation
WER (Word Error Rate)	Agent 1 — Transcription	~8–12%	Below 10% is considered production-ready for clean speech. Degrades on noisy audio.
Flesch Reading Ease improvement	Agent 2 — Simplification	+16.6 pts avg	Simplified text is measurably easier to read than the original transcript.
Compression Ratio	Agent 3 — Summarization	0.30 avg	Summaries are approximately 30% the length of the original transcript.
ROUGE-L (recall)	Agent 3 — Summarization	0.42	Moderate unigram overlap with reference. Expected for abstractive models.

8.2 Where Results Were Weaker — and Why

NOTE: Poor results are not hidden here. Transparency about limitations is part of rigorous evaluation.

- **WER 18–25% on noisy audio:** Whisper-base is sensitive to background noise and overlapping speech. Clips with wind, music, or multiple speakers produced significantly higher error rates. Using Whisper-small or Whisper-medium reduces this at the cost of ~2x inference time.
- **FLAN-T5 repetition on long chunks:** When chunk size exceeded 100 words, FLAN-T5-base sometimes repeated phrases in its output. Reducing chunk size to 60 words corrected this but increased total inference time by ~20%.
- **DistilBART hallucination on short clips:** Clips shorter than 80 words caused DistilBART to occasionally generate plausible-sounding but incorrect details not present in the transcript. This is a known failure mode of abstractive models on very short inputs. Mitigated by passing very short clips through unchanged.
- **Slow inference on CPU:** End-to-end pipeline takes 60–120 seconds on CPU for a 2-minute clip. Acceptable for a demo but not for real-time use. GPU inference would reduce this to under 10 seconds.

9. Deployment

9.1 Local Deployment (VS Code)

Steps to run the project on any Windows / Mac / Linux machine:

```
# 1. Create and activate virtual environment
python -m venv venv
venv\Scripts\activate          # Windows
source venv/bin/activate       # Mac / Linux

# 2. Install ffmpeg (system-level, required by Whisper)
brew install ffmpeg            # Mac
sudo apt install ffmpeg        # Linux / WSL

# 3. Install Python dependencies
pip install -r requirements.txt
python -c "import nltk; nltk.download('punkt')"

# 4. Launch Streamlit UI
streamlit run ui/streamlit_app.py  # opens http://localhost:8501
```

9.2 Cloud Deployment (Google Colab)

The project also runs in Google Colab for demo or sharing purposes. Upload `voxa_team_streamlit.zip`, extract it, install dependencies, and run `streamlit run ui/streamlit_app.py` &. Use ngrok or Colab's port forwarding to expose the UI.

9.3 Deliverables

Deliverable	Status	Details
Working web interface	Delivered	Streamlit app at <code>ui/streamlit_app.py</code> — upload audio, view 3 outputs, download results
GitHub repository	Delivered	Final code version with all agent files, <code>config.py</code> , and <code>requirements.txt</code>
Trained models	Delivered	FLAN-T5 and DistilBART checkpoints in <code>models/</code> folder, loaded via <code>use_pretrained:False</code> in <code>config.py</code>
Dataset access URL	Delivered	https://huggingface.co/datasets/ajibawa-2023/Audio-Children-Stories-Collection
MLops tool	Delivered	Hugging Face Trainer for training; Hugging Face Hub for model storage and versioning
Final report	Delivered	This document

10. Results & Conclusion

10.1 What Worked

- Whisper achieved WER of ~10% on clean speech — acceptable for educational content where speakers are clear and audio quality is controlled.
- Flesch Reading Ease improved by an average of +16.6 points after simplification — a statistically meaningful shift toward accessible language.
- Compression ratio of 0.30 confirms that summaries convey key points at roughly a third of the original length.
- The multi-agent architecture allowed 5 team members to work on separate files simultaneously without merge conflicts.
- The `use_pretrained` flag in `config.py` enabled smooth switching between pretrained fallbacks and trained models during development.
- End-to-end pipeline runs in under 3 minutes on CPU for a typical 2-minute audio clip.

10.2 What Didn't Work / Limitations

- WER degrades significantly (18–25%) on clips with background noise. Whisper-base is insufficient for real-world noisy environments without audio denoising.
- CPU-only inference is too slow for real-time or high-throughput use. A GPU reduces inference time by approximately 6–10x.
- DistilBART can hallucinate on very short inputs (<80 words). The model needs a minimum content length to produce reliable summaries.
- FLAN-T5 simplification quality is uneven on very technical vocabulary. The model sometimes produces grammatically correct but semantically thin simplifications.

10.3 Conclusion

Voxa demonstrates that a structured multi-agent pipeline can convert spoken audio into three layers of accessible text — verbatim, simplified, and summarized — reliably and offline, using open-source models that run on a standard laptop.

The project's primary contribution is architectural: separating transcription, simplification, and summarization into three independently testable agents enabled parallel team development, isolated debugging, and clear grading boundaries. The configuration-driven design meant that the entire system's behavior could be adjusted from a single file without touching agent logic.

The most important lesson from the project: audio quality has a larger effect on final output quality than model size. Improving the preprocessing pipeline (better denoising, speaker diarization) would yield more improvement than upgrading from Whisper-base to Whisper-medium.